Laboratorio 2.3: RETO DE INTEGRACIÓN — SISTEMA RESILIENTE DE RESEÑAS

1. Contexto y Objetivos

El objetivo de este reto es construir un **Sistema de Reseñas de Productos** que sea inmune a la pérdida de datos. La aplicación debe demostrar un manejo profesional del ciclo de vida de Android 16, asegurando que el usuario no pierda su progreso ante rotaciones, falta de memoria o navegación accidental.

2. Requisitos Previos

- Proyecto basado en la estandarización del **Laboratorio 1** (API 36, Java 21).
- Conocimiento de las herramientas de persistencia vistas en los **Laboratorios 2.1 y 2.2**.

3. Descripción del Reto

Crear una aplicación que permita al usuario calificar un producto (del 1 al 5) y escribir un comentario. Se deben cumplir estrictamente las siguientes reglas de persistencia:

1. **El Puntaje (Rating):** Debe sobrevivir a la rotación de pantalla.
2. **El Comentario:** Debe sobrevivir a la rotación de pantalla y a la muerte del proceso (Process Death).
3. **El Borrador:** Al hacer clic en un botón "Guardar Borrador", el texto debe persistir incluso si la aplicación se cierra y se vuelve a abrir.
4. **Navegación:** Impedir el cierre de la app mediante el gesto "Atrás" si el usuario tiene texto escrito en el comentario.

4. Paso a Paso Detallado

Paso 1: Configuración de Arquitectura

1. **Copiar** la carpeta del proyecto del Laboratorio 1 y **renombrarla** a RDA1_Lab8_RetoFinal_NombreApellido.
2. **Limpiar** archivos temporales (carpetas .gradle y build).
3. **Renombrar** el proyecto en settings.gradle.kts a ReviewResilience.
4. **Añadir** las dependencias necesarias para ViewModel, Compose LiveData y DataStore en el archivo build.gradle.kts del módulo app.

Paso 2: Implementación de la Capa de Datos

1. **Crear** una clase para gestionar el almacenamiento en disco (DataStore).

2. **Definir** un flujo de lectura y una función de escritura para un valor de tipo String llamado "draft_content".

Paso 3: Definición del ViewModel

1. **Crear** la clase ReviewViewModel heredando de ViewModel.
2. **Integrar** SavedStateHandle en el constructor para gestionar la persistencia del comentario.
3. **Declarar** una variable para el puntaje numérico que sea observable por Compose.
4. **Implementar** las funciones necesarias para actualizar el estado y disparar el guardado en disco mediante corrutinas.

Paso 4: Construcción de la Interfaz y Control de Ciclo de Vida

1. **Abrir** MainActivity.kt.
2. **Registrar** mediante Log.d cada uno de los 6 eventos del ciclo de vida (onCreate, onStart, onResume, onPause, onStop, onDestroy).
3. **Diseñar** la interfaz usando Column, un componente para el puntaje (puede ser un Slider o botones) y un OutlinedTextField para el comentario.
4. **Configurar** un BackHandler que esté activo únicamente cuando el campo de comentario no esté vacío.

Paso 5: Pruebas de Validación

1. **Ejecutar** la aplicación en el emulador.
2. **Realizar** la prueba de rotación: Verificar que el puntaje y el texto no cambien.
3. **Realizar** la prueba de "Muerte de Proceso": Activar "Don't keep activities" en opciones de desarrollador, escribir un comentario, ir al inicio y volver. El comentario debe permanecer.
4. **Realizar** la prueba de "Cierre Total": Escribir, pulsar "Guardar Borrador", cerrar la app desde recientes y abrirla de nuevo. El texto guardado debe reaparecer.

5. Resultados Esperados

- Registro completo en **Logcat** de la destrucción y recreación de la Activity al rotar.
- Restauración automática de datos en los tres escenarios de prueba (rotación, proceso y disco).
- El gesto de "Regreso" no debe cerrar la app si hay un comentario en curso.

6. Entregables

- **Repositorio de GitHub:** Con un historial de al menos 3 commits descriptivos.
- **Informe PDF con mínimo:**

- Capturas de pantalla de la ejecución de las 3 pruebas de validación y del Logcat.
- Copia del código como Anexo
- Respuestas a preguntas
- Demás puntos que debe contener el informe como conclusiones, recomendaciones, etc.

7. Criterios de Evaluación

No olvide revisar la rúbrica de evaluación, en el EVA, para complementar su informe.